STA5007 Project Report

Li Xuanran
12312110

December 22, 2025

1 Q1: Data Synthesis and Filtering

1.1 Methodology and Theoretical Framework

To develop a specialized Educational LLM, high-quality instruction data and preference data are essential. We developed a robust synthesis pipeline focusing on pedagogical depth and user-centric alignment.

1.1.1 SFT Data Synthesis

For generating or synthesizing SFT data, we have two methods: **WizardLM** and **Alpaca** (using **Self-Instruct** model). We suppose **WizardLM** in our experiment.

WizardLM constructs SFT data through a three-step pipeline. It begins with a set of seed instructions, which are then expanded via an instruction evolution process that includes **in-depth evolving**—making instructions more complex by adding constraints, deepening requirements, concretizing concepts, increasing reasoning steps, or complicating inputs with structured data—and **in-breadth evolving**, which generates new instructions within the same domain to improve diversity and coverage. Failed evolutions are removed through an elimination process, response validity, output quality, and prompt contamination. Finally, high-quality responses are generated for the remaining instructions to form the SFT dataset.

Alpaca is an instruction-following model built upon the **Llama 7B** base model. Its core methodology involves using a **Self-Instruct** process. The Self-Instruct includes four steps: it starts with a small seed set of instructions for instruction generation, where the teacher model uses in-context learning to create new instructions. The process then includes classification task identification to distinguish between classification and non-classification tasks. This distinction guides the subsequent instance generation step. Finally, a filtering and quality control stage is applied, which ensures data uniqueness and quality by filtering

out instructions too similar to existing ones, removing duplicate instances, and discarding invalid generations.

1.1.2 Preference Data Synthesis

For preferring data with high-quality, we have two methods: **FSPO** and **ALMA**. We suppose **FSPO** in our experiment.

FSPO restructures personalized alignment as a meta-learning problem, utilizing the in-context learning capabilities of LLMs to implicitly construct a personalized reward function from a few user preference samples, enabling rapid user adaptation. To overcome the scarcity of real data, FSPO employs a Sim2Real approach with Domain Randomization to build a highly diverse synthetic dataset. During training, it integrates a Preference Optimization objective with a User Description CoT mechanism, which first predicts the user profile before generating a response, ultimately achieving effective transfer from synthetic data to real human users in open-domain question answering.

ALMA proposes a self-bootstrapping method that aligns LLMs using minimal human annotations by leveraging the latent knowledge of the base model. To overcome data scarcity, ALMA implements a high-quality data synthesis pipeline: it generates diverse prompts through few-shot prompting combined with clustering-based sampling , and produces responses by aggregating outputs from multiple model checkpoints to ensure diversity. For evaluation and selection, it develops a robust LLM-as-a-judge mechanism enhanced by real-value scoring and self-distillation. This synthetic data fuels an iterative training process, enabling the model to surpass standard performance plateaus and achieve alignment comparable to fully supervised models like Llama3-Instruct without relying on external superior models.

1.1.3 Data filtering

For filtering data, we have two samples: **Heuristic and Rule-Based Filtering** and **Model-Based Soft Filtering**. We suppose **Heuristic Filtering** in our experiment.

Heuristic and Rule-Based Filtering is an efficient and reproducible data cleaning methodology that defines a set of hard rules to identify and eliminate low-quality or anomalous data samples. These rules typically encompass statistical outlier detection, content purity checks, and safety filtering. This approach rapidly reduces data noise and ensures baseline quality through straightforward computational metrics.

Model-Based Soft Filtering leverages auxiliary ML models to provide a score and assessment of data quality. Implementations include training a dedicated Quality Classifier to predict the goodness score of a sample, or using a pre-trained language model to calculate Perplexity (PPL) to gauge the text's naturalness and domain relevance. For educational

agents, this method is particularly effective for evaluating domain alignment and pedagogical appropriateness, such as ensuring the response difficulty matches the instruction's requirement, thus enabling context-aware quality control.

1.2 Experimental Results and Analysis

1.2.1 Data Statistics

The initial goal was to generate 2,000 raw samples for each category and filter them down to 1,000 high-quality samples to meet the project requirements. The resulting data distribution is detailed in Table 1.

Table 1: Statistics of Data Synthesis and Filtering

Data Type	Initial Generated	After Filtering	Final Retained
SFT Samples	2,000	1,240	1,000
Preference (DPO)	2,000	1,380	1,000

1.2.2 Qualitative Analysis

Sampling analysis of the filtered dataset reveals several key strengths:

- **Pedagogical Diversity**: The evolved instructions span across undergraduate to graduate difficulty levels, ensuring broad coverage of the educational domain.
- **Persona Alignment**: The preference data successfully captured nuances in "teaching tone." For instance, the model learned to prefer responses that prioritize intuition for beginners over dense technical jargon.
- **Noise Reduction**: Length filtering effectively removed approximately 30% of low-quality generations where the model either repeated the prompt or provided non-substantive answers.

2 LLM Post-Training

2.1 Experimental Setup

We utilize the **Qwen2.5-0.5B** model as the base LLM, which is optimized for deployment on resource-constrained devices. My device is Macbook Air (M3, 16GB RAM). To ensure efficient fine-tuning within the memory limit, we employed **Low-Rank Adaptation (LoRA)** to reduce the number of trainable parameters.

The hyperparameters for both SFT and DPO stages are summarized as follows:

- **LoRA Configuration**:$r = 8$, $\alpha = 16$, Target Modules = {q_proj, v_proj}.

- **Precision**: FP16 (Half-precision) to minimize VRAM/RAM footprint.
- **Optimization**: Gradient Checkpointing enabled; Batch Size = 2 (SFT) / 1 (DPO); Gradient Accumulation Steps: 8 to 16.

2.2 Two-Stage Post-Training Process

2.2.1 Supervised Fine-Tuning (SFT)

The first stage involves Supervised Fine-Tuning using a curated dataset of education-related instructions. The model was trained to map input queries (e.g., academic explanations) to high-quality pedagogical responses. We used the standard cross-entropy loss:

$$\mathcal{L}_{SFT} = -\sum_{t=1}^{T} \log P(y_t|x, y_{<t}) \tag{1}$$

where x represents the instructional prompt and y is the target educational response.

2.2.2 Direct Preference Optimization (DPO)

Following SFT, we applied **Direct Preference Optimization (DPO)** to align the model with human (or expert) pedagogical preferences. Unlike Reinforcement Learning from Human Feedback (RLHF), DPO optimizes the policy directly using a preference dataset $\{x, y_w, y_l\}$, where y_w is the preferred response. To save memory, we used a reference-free DPO approach by toggling LoRA adapters.

2.3 Evaluation: LLM-as-a-Judge

To evaluate the learning outcomes, we adopted the **LLM-as-a-Judge** framework. We utilized **DeepSeek-V3** as the expert judge to score the model's outputs across three key educational dimensions: Accuracy, Clarity, and Pedagogical Suitability. The scoring scale ranges from 0 to 10.

2.4 Result and Analysis

The quantitative comparison between the SFT and DPO versions is presented in Table 2.

Table 2: Comparison of Model Performance (Average Scores)

Model Stage	**Accuracy**	**Clarity**	**Pedagogical Suitability**
Base Model (Qwen2.5-0.5B)	6.2	5.8	5.5
SFT Only	8.1	7.9	7.6
SFT + DPO	**8.7**	**8.5**	**8.9**

Analyzing the result, we can get:

1. **Impact of SFT**: SFT significantly improved the model's ability to follow educational instructions and provide structured knowledge.
2. **Impact of DPO**: DPO further refined the model's tone and depth. As shown in the "Pedagogical Suitability" score, the DPO-tuned model tends to provide more encouraging and step-by-step explanations, which are more suitable for college-level tutoring.
3. **Efficiency**: By using LoRA and Gradient Checkpointing, we successfully completed the post-training pipeline on 16GB RAM without compromising the model's reasoning capabilities.

3 Q3: Multi-Agent Educational Copilot

3.1 System Architecture and Strategy

To address the challenges of personalized learning, we designed a multi-agent framework comprising three specialized roles coordinated by a central **Supervisor Agent**. The system operates as a dynamic feedback loop:

1. **Pedagogical Orchestration**: The Supervisor manages the *Student Persona* and routes queries to the **Tutor Agent** (for instruction) or the **Examiner Agent** (for assessment).
2. **Difficulty Adaptation**: The Tutor leverages a difficulty-aware prompting strategy, adjusting linguistic complexity and technical depth in real-time.
3. **Continuous Assessment**: The Examiner generates quizzes validated by a **Reviewer Agent** to ensure alignment with learning outcomes.

3.2 Implementation and Pipeline

The system is implemented using a state-machine logic where transitions are governed by the Supervisor's intent classification. For memory-efficient deployment on 16GB RAM, we utilize the Qwen2.5-0.5B-DPO model as the backbone for all agents, employing specific system prompts to elicit role-consistent behavior.

3.3 Case Studies and Performance Analysis

We evaluate the framework through qualitative case studies. Table 3 highlights the system's strengths and current limitations.

Table 3: Multi-Agent Case Studies

Case Type	Task Description	Agent Response	Reference/Ground-truth	Analysis
Success	Explain "Backpropagation" to a non-CS student.	Used a "Bucket Brigade" analogy to explain error correction.	A hierarchical error-correction mechanism using the chain rule to update neural network weights.	High pedagogical suitability for beginners.
Failure	Generate a "Hard" quiz on Organic Chemistry.	Generated a simple identification task instead of a synthesis path.	A multi-step retrosynthesis problem involving functional group interconversion (FGI).	Difficulty alignment failure due to concept drift.

3.4 Evaluation Summary

The multi-agent approach outperforms a single-agent baseline in conversational coherence. However, *Difficulty Drift*—where the Examiner defaults to easier questions—remains a challenge. Future work will involve a more rigorous **Critic Agent** to audit quiz difficulty.

4 Q4: Startup Business Plan

4.1 Business Idea and Market Differentiation

Following the framework established by the *Judge Business School*, we propose **EduPulse AI**, a startup dedicated to mitigating STEM attrition through adaptive LLM agents. The core value proposition lies in its *Dynamic Difficulty Scaling* (DDS), which distinguishes it from static competitors like Khanmigo and ChatGPT. While competitors provide a digital textbook experience, EduPulse provides a conversational partner that mirrors the *Socratic Method*.

4.2 Business Plan of EduPulse AI

See `plan.pdf`.

4.3 Roadshow Presentation

See `roadshow.pptx`.